



Linux Compatible OS

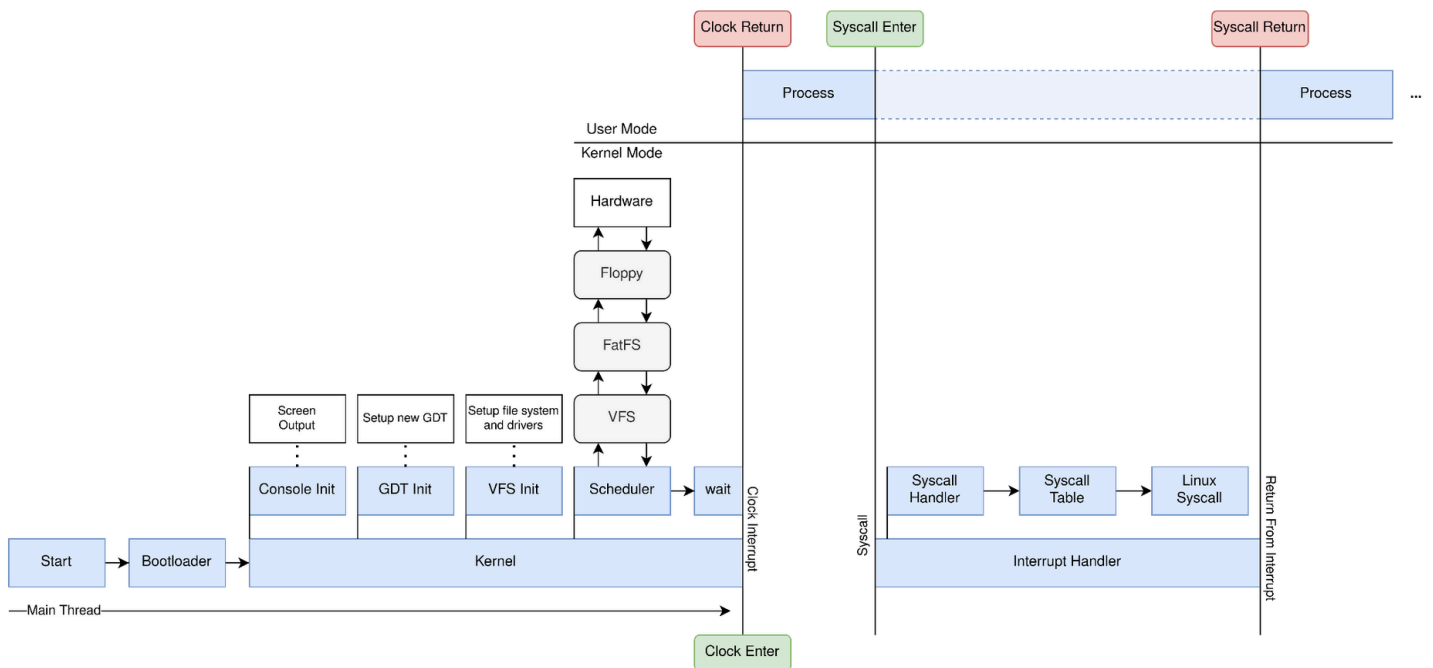
Full documentation

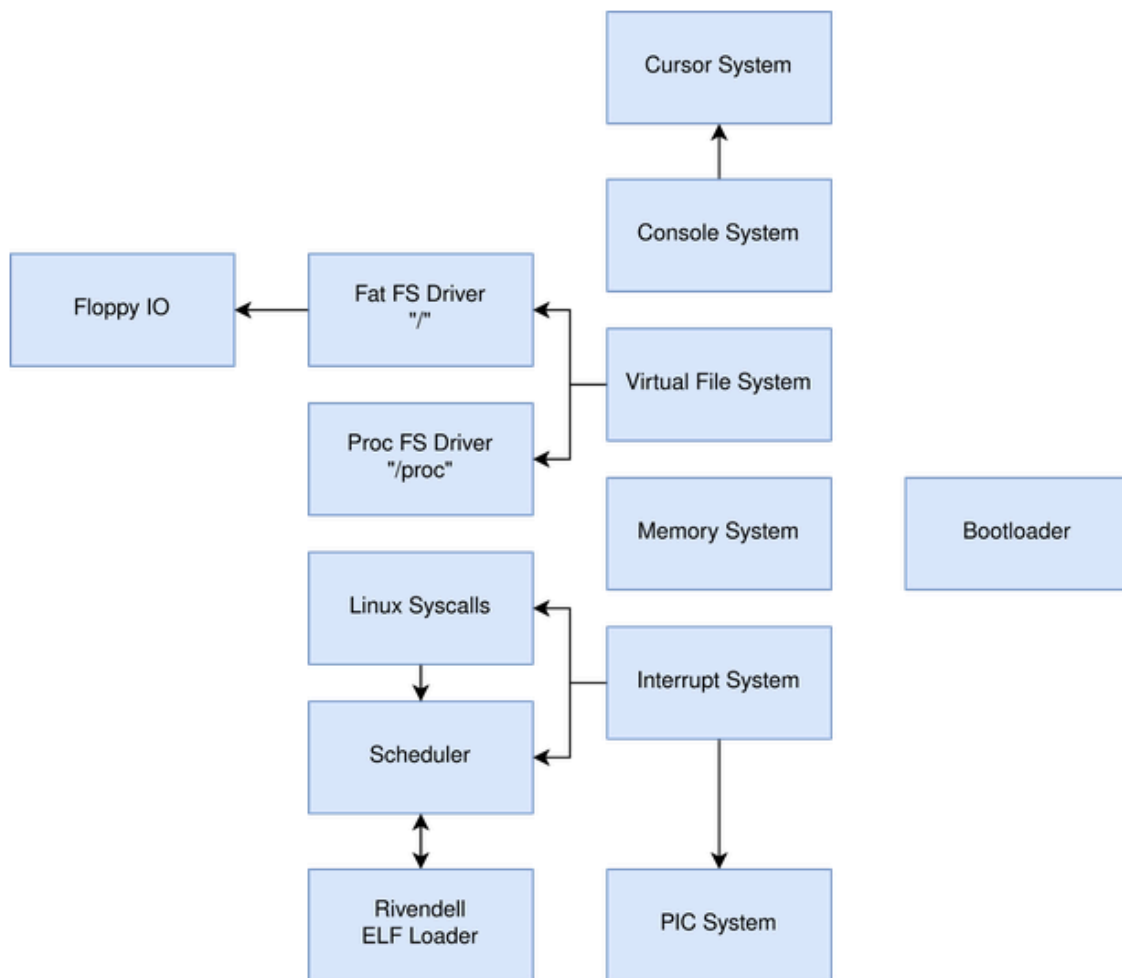
Authors:

- Mateusz Nurczyński
- Mateusz Tkaczewski
- Maksymilian Goryszewski

Table Of Contents

- Capabilities
- Technical Summary
 - Interrupt System
 - Memory Allocator
 - Filesystem
 - Floppy Support
 - Scheduler





Capabilities

The NEOS is an open-source (GPL 3.0), 32-bit x86, multi-tasking operating system. It implements Linux syscall ABI, as well as operates on the Linux-compatible ELF executable file format.

NEOS in its current form (commit [2e49244](#)) is capable of starting linux ELF executables as long as they do not rely on dynamic linking. The process can then run as long as an unsupported syscall is not invoked. In theory the system should support C programs that utilize the Standard C Library as long as it is statically linked with the application, however, due to an currently unresolved bug, the standard library fails to initialize and crashes the system before control is passed to the main() function. System supports the running of multiple processes at the same time, though only on one CPU thread.

The processes are started in 32 bit mode, and are memory protected by placing them in ring 3. The range of supported syscalls (See table below) allows the programs to operate on files in the file system, and perform some other minor actions, like querying their own pid, gid, or system uname.

The system also provides partial support of the Linux ProcFS, allowing processes to query their own (and other's) executable path, and current working directories.

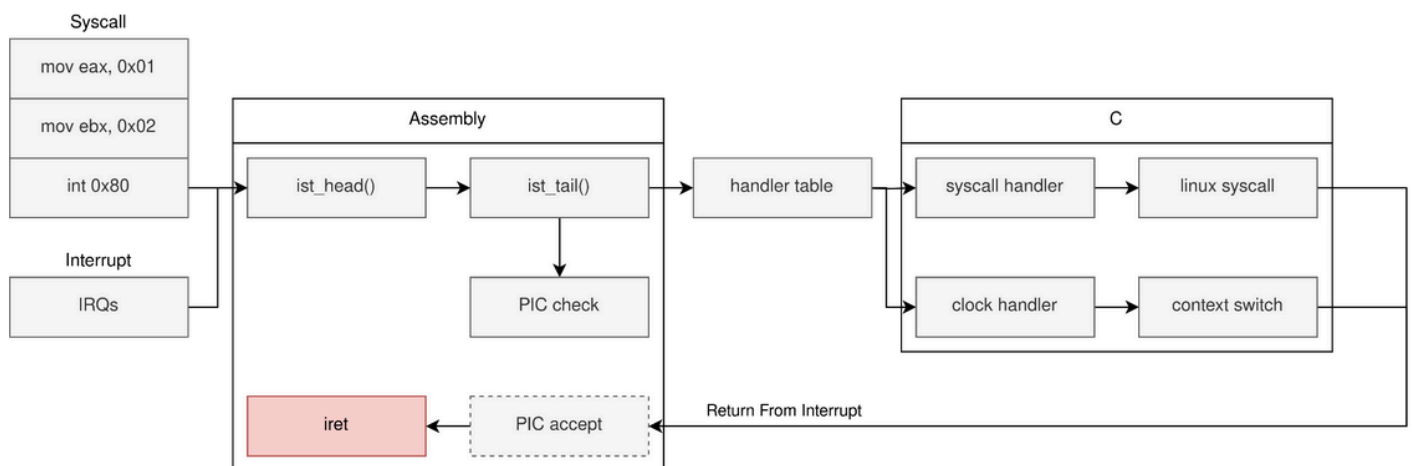
Syscall Name	Syscall Description
sys_write	Write to file
sys_open	Open or create file
sys_read	Read from file
sys_creat	Create file
sys_lseek	Seek in file
sys_openat	Open or create file (CWD invariant)

sys_mkdir	Create directory
sys_readlink	Read link as path
sys_getdents64	Read directory contents (64 bit)
sys_getdents	Read directory contents
sys_old_readdir	Read directory contents (deprecated)
sys_stat	Get file info
sys_fstat	Get file info
sys_lstat	Get file info
sys_newstat	Get file info
sys_newlstat	Get file info
sys_newfstat	Get file info
sys_stat64	Get file info
sys_lstat64	Get file info
sys_fstat64	Get file info
sys_close	Close file descriptor
sys_unlink	Delete file
sys_unlinkat	Delete file (CWD invariant)
sys_rmdir	Remove directory
sys_mkdirat	Create directory (CWD invariant)
sys_chdir	Change working directory
sys_getcwd	get CWD
sys_getpid	Get PID
sys_uname	Get system name
sys_olduname	Get system name
sys_newuname	Get system name
sys_brk	Change process resource limit
sys_exit	End process

Interrupt System

The interrupt system is written in assembly, all incoming interrupts (be it a one from IRQ or INT come to a unique `ist_head<int_number>` function that are generated using a macro. That pushes the interrupt number onto the stack, if necessary, pushes an additional DWORD 0 to account for some interrupts pushing an additional error code onto the stack, and then jumps to a common handler `isr_tail`. That common function handles communication with PIC (Programmable Interrupt Controller), discards spurious interrupts, invokes common C handler (that handles waiting for interrupts) and invokes the interrupt-specific C handler from handler array.

When that handler returns `isr_tail` signals the PIC and returns from interrupt. As shown on the diagram below:



NEOS Memory Allocator

The NEOS memory allocator is a highly efficient multi-purpose memory allocator, intended to address all of the NEOS kernel's memory allocation needs.

It is based on the buddy system and utilizes a binary tree control structure, allowing for $O[\log(n)]$ worst-case time complexity of all successful memory allocations, as well as $O[\log(1)]$ worst-case time complexity for all non-successful memory allocations.

Usage

Setup

In order to properly initialize the NEOS Memory Allocator, do the following:

1. Call the *kset* function, which will initialize the allocator control structures
2. Make all the necessary *kmres* calls, to exclude memory areas, that you do not want to be available for allocation. Make sure, that you make all *kmres* calls, before the first *kmalloc* call.

Functions

Afterwards, you may freely use the *kmalloc*, *kfree*, *krealloc*, and *kmsz* functions, as needed.

Detailed documentation on all the aforementioned functions is available in the header file.

Control structure

Overview

Memory is allocated in blocks, 1024 bytes each. Every allocated memory area is X blocks, where X is a power of 2 (so 1kB, 2kB, 4kB etc.).

Binary tree representation

The main allocator control structure is a binary tree, stored in arrays. Each tree level (level 0 being the leaves), has its own array. There is one fixed-size array, storing pointers to all the tree levels. Structure of this array is presented below.

Tree level	Size, in number of nodes (4 bytes)	Pointer (4 bytes)
0	<offset=0>	<offset=4>
1	<offset=8>	<offset=12>
...	...	...
31	<offset=248>	<offset=252>

There can be a maximum of 32 tree levels.

Every node on level 0 represents a single block. Every node on level 1 represents a 2-block area. Every node on level 2 represents a 4-block area, etc.

Every node on level 0 is 1-bit long, every node on level 1 is 2-bit long, every node on level 2 is 3-bit long, etc. Nodes are packed in the array, but rounded up to full bytes (so a tree level with 4 3-bit nodes will take 12 bits, rounded up to 2 bytes).

If M is the tree level, then:

$$\text{represented memory area [blocks]} = 2^M$$

$$\text{node size [bits]} = M+1$$

Node structure

Let's say a node has M bits (on tree level $M-1$).

If bit 0 (the least significant bit) is set to 1, it means that a single-block area cannot be allocated within the memory space represented by this node (so the entire 2^{M-1} block space is fully allocated). If it is set to 0, it means that at least one block in that space is free.

If bit 1 is set to 1, it means that nowhere in that memory space, a 2-block area can be allocated. If it is set to 0, then it can be allocated.

If bit 2 is set to 1, it means that nowhere in that memory space, a 4-block area can be allocated.

Therefore, assuming tree level 2:

111 - the 4-block memory space is fully allocated.

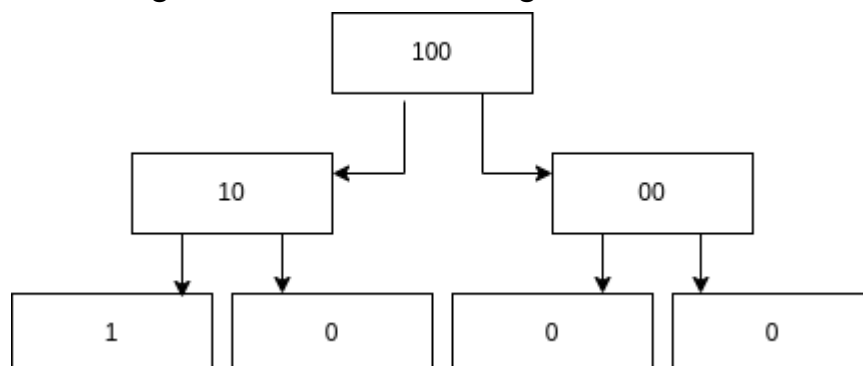
110 - at least one single-block memory area can be allocated here.

100 - at least one two-block memory area can be allocated here.

000 - The entire 4-block memory space is free.

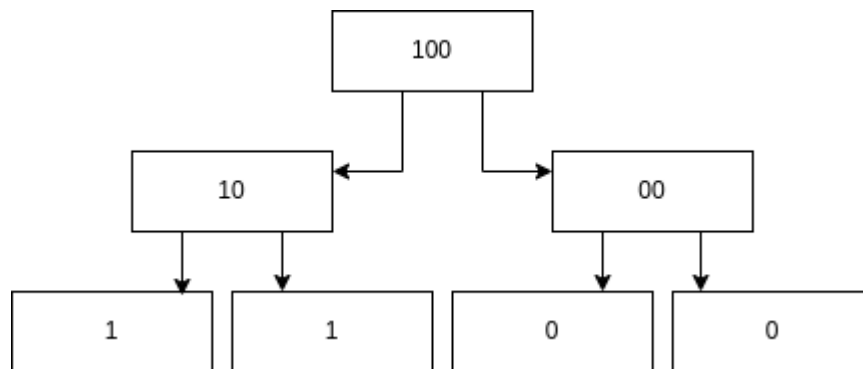
Allocating a single-block area

Assuming we have the following tree structure:



We can see that block 0 is already allocated (because node 0 on tree level 0 is set to 1). To indicate that a 2-block area cannot be allocated three, but a 1-block area can, block 0 on level 1 is set to 10.

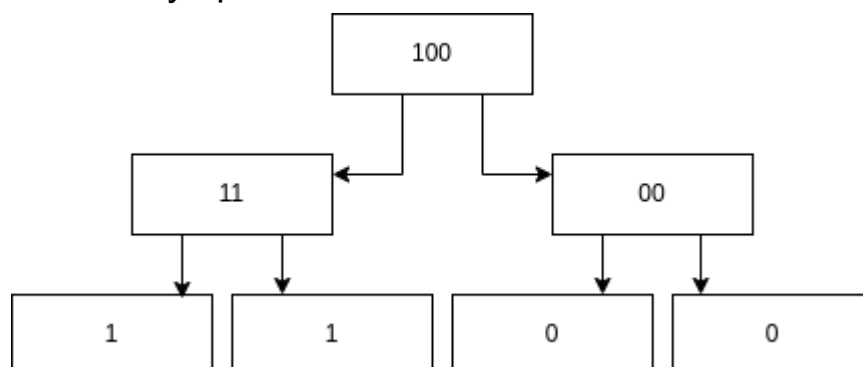
Now we will allocate another 1-block area:



As you can see, the value of the node 1 on level 0 was updated to 1, indicating that this 1-block area was allocated.

Now we need to make sure that the rest of the tree is up to date (currently it is not, since node 0 on level 1 is set to 10, which would mean that a 1-block area can still be allocated here, while it can't).

We do this by performing, what we will refer to as the *buddify* operation recursively up the tree:



We changed the value of node 0 of level 1 accordingly, the root (node 0 on level 2) need not be changed here, since allocating a 2-block area is still possible.

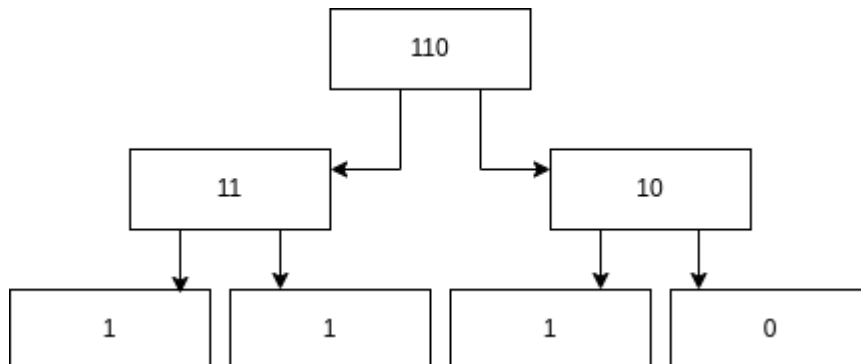
We will now allocate another 1-block area.

Notice how, starting our search at the root, we only need to check the root's 2 children to know where the 1-block area can be allocated.

The left child corresponding bit is set to 1, so we know that it cannot be allocated in this branch.

Therefore we go down the other branch.

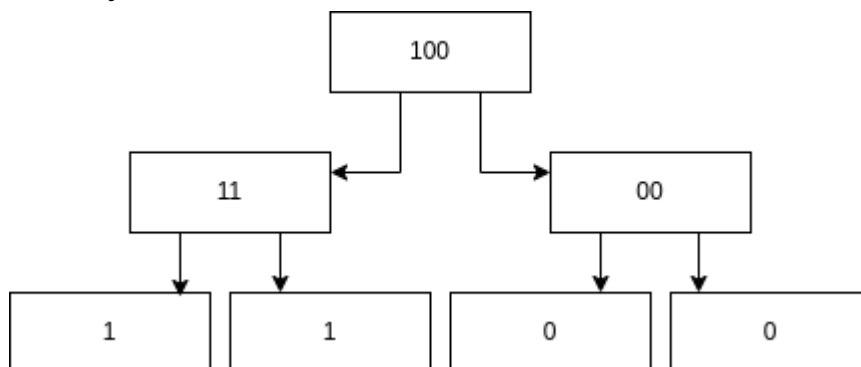
We set node 2, on level 0 to 1 and execute buddify recursively up the tree.



Buddify has changed the value of node 1, level 1 to 10 (since allocating a 2-block area is no longer possible in that branch, while allocating a 1-block area is still possible). We also changed the value of node 0, level 2 to 110 for the same reason.

Freeing a single-block memory area

To free an area, we change the value of a level-0 node to 0 and execute buddify:



Allocating a multiple-block memory area

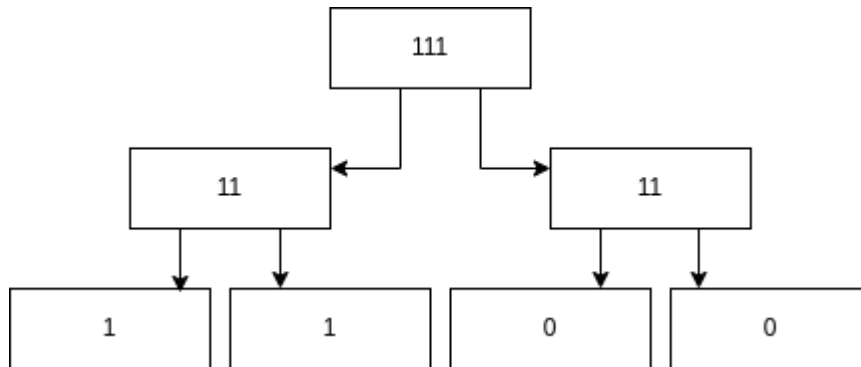
We will now allocate a 2-block memory area.

We start the search at the root, just as in previous examples.

This time, however, we will stop at level 1 (because $2^1 = 2$), instead of going down all the way to level 0 (which we did, because $2^0 = 1$).

If we were allocating an 8-block area, we would stop the search at level 3 (since $2^3 = 8$).

We change this node's value to all 1s and execute buddify:



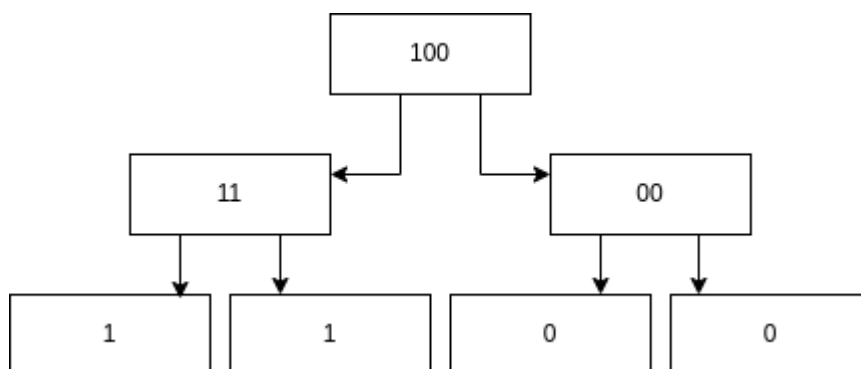
Notice, how nodes 2 and 3 on level 0 are both remain set to 0.

This is how we know that the tree branch on the left (node 0, level 1) has two single-block areas allocated, while the branch on the right (node 1, level 1) has 1 two-block area allocated.

Notice, that if we now tried to allocate any other area, we would only need to check the value of the root, to know it is not possible.

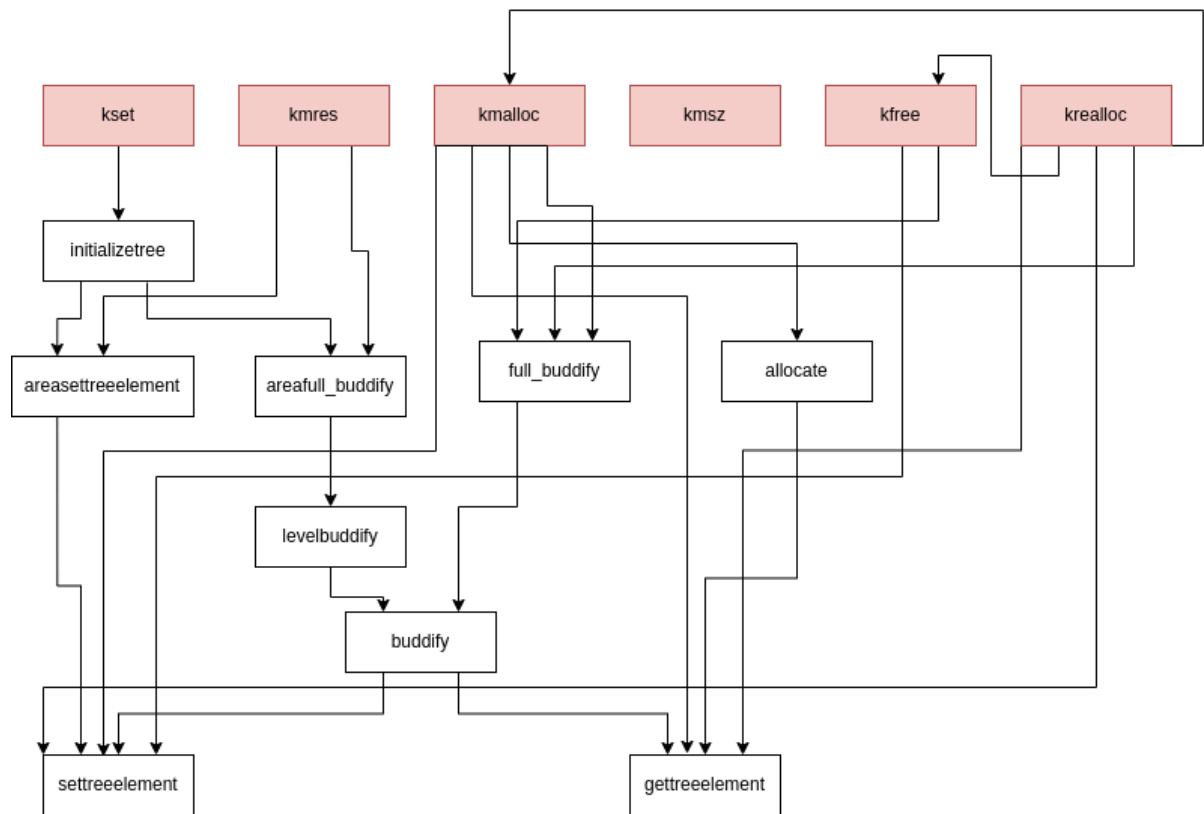
Freeing a multi-block area

To free an area (of any size) you set the value of 0-level node to 0 (if we're freeing a multi-block area it is already set to 0, this this changes nothing) and execute buddify (the exact same procedure as with the single-block area):

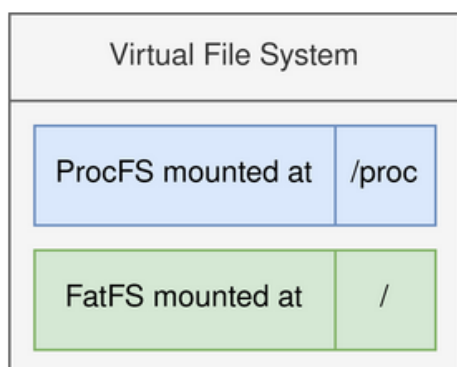
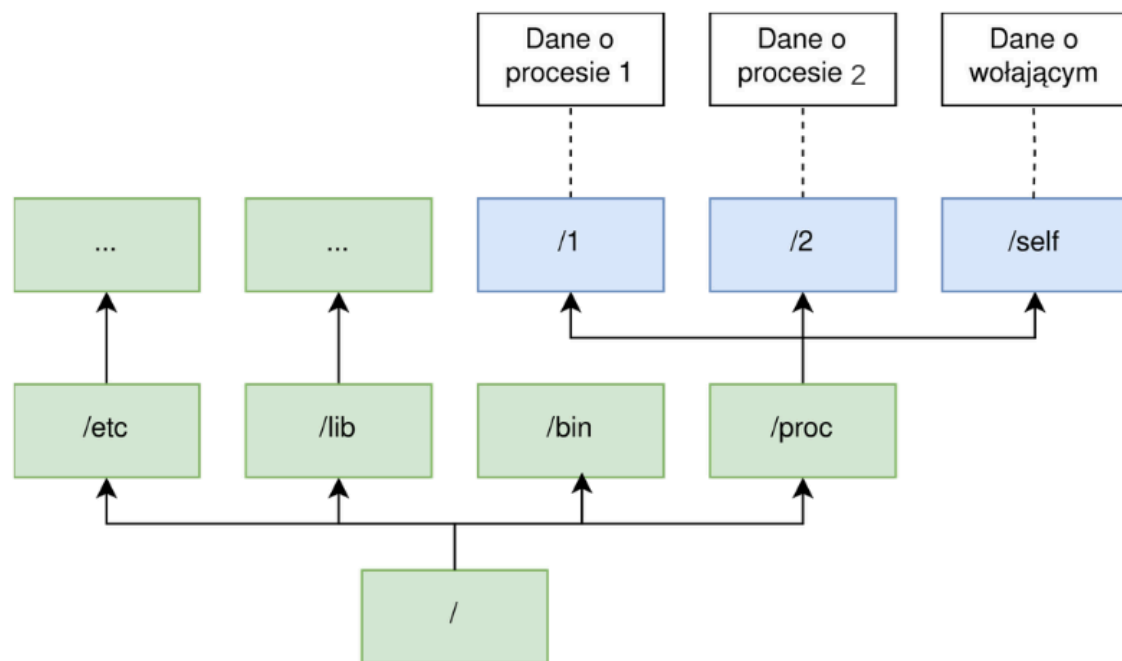


Implementation

Below is a diagram, displaying the structure of internal function calls in the allocator. Functions available to call from the outside were marked as red.



Filesystem



FAT32

NEOS FAT32 driver provides full support for all filesystem operations. The filesystem is utilized as a main data storage.

PROCFS

NEOS PROCFS driver provides partial support for all filesystem operations and supports some of the paths outlined in Linux /proc.

Floppy Disk

NEOS in its current form utilizes floppy disks as both system and data storage. The floppy driver in the system consists of easy to use functions allowing floppy disk controller initialization, reading and writing data. The addressing of data is independent from the physical data layout on the disk. Only the address of the first byte and data size is required to perform read / write operation.

Scheduler

NEOS scheduler is responsible for process loading, storing process information, queue management (using the round-robin algorithm) and context switching. It also manages the GDT table, through the GDT subsystem.

Process structure in memory

Each process has:

1. An entry in the General Process Table (a dynamic array of Process Descriptors), where process metadata is stored (crucially, this includes the current address of the process Interrupt Stack).
2. A working area in memory, where code and data is stored. Main Process Stack is located at the end of this area.
3. A Process Interrupt Stack.

4. A TSS table, storing address and segment descriptor of the Process Interrupt Stack
5. 3 GDT entries: X (code segment), X+1 (data segment) and X+2 (TSS selector)
6. An array of File Descriptors, for the filesystem.

Stacks

If an interrupt is invoked, when CPL=0 the interrupt handler will use the current stack.

If an interrupt is invoked, when CPL=3 the stack will be automatically switched to the stack address stored in the current process TSS. ESP and SS registers (pointing to the old stack) will be pushed on the new stack (before pushing EIP and CS, which is done on every interrupt regardless of the CPL).

For that reason, when CPL=3, the TSS always has to point to the stack that an interrupt can use. Since the Main Process Stack is in use constantly, it is not possible for the TSS to always store an up-to-date pointer to this stack.

Therefore it is necessary to have a separate stack, just for interrupts.

In NEOS, every process has its own Interrupt Stack (aside from the Main Stack), since values of general-purpose-registers are stored there, when the process is idle.

When the process is running its Interrupt Stack is always empty and the process uses the Main Stack.

When an interrupt is invoked, the processor automatically switches to the Interrupt Stack and pushes there ESP and SS pointing to the Main Stack. Then, upon returning from the interrupt, those values are automatically popped from the stack by the IRET instruction. Therefore the processor goes back to the Main Stack.

Segmentation

NEOS does not use paging, only segmentation.

Every process has 2 segments on privilege level 3: data segment (put in SS, DS, ES etc.) and code segment (put in CS).

Base and limit values of both segments are exactly the same (so the process has unchecked access to its own memory area).

Kernel has 2 segments (data and code segment) on privilege level 0, spanning the entire 4GB address space.

Context Switching

Context switching is done at every clock interrupt.

Context switching steps (Process1 is the current process, Process2 is the new process):

Process1's TSS selector number is in the TR register, segment registers are set to Process1's segments.

1. A clock interrupt is invoked. ESP and SS are set to values provided by the current TSS (so Process1's Interrupt Stack - the SS is set to the kernel data segment). Process1's Main Stack is left intact - old values of SS and ESP are pushed onto the Interrupt Stack, alongside the return address (EIP) and return CS.
2. Common Interrupt Handler pushes general-purpose registers onto Process1's interrupt stack
3. Since this is a clock interrupt - control is passed to the clock interrupt handler. This function takes the value of Process2's interrupt stack from the General Process Table (which was stored there either during a previous context switch or during process creation) and puts it in the ESP register, stores the old value of the ESP (so the pointer to Process1's interrupt stack to the General Process Table). We are now on Process2's Interrupt Stack.

4. We take Process2's TSS selector number from the General Process Table and put it into the TR register.
5. We pop general-purpose registers (which were stored on Process2's Interrupt Stack either during a previous context switch or during process creation).
6. We execute the IRET instruction, which pops Process2's CS and return address, as well as Process2's SS and ESP (pointing to the Process2's Main Stack). Thus IRET jumps to Process2 and switches to its stack.

Note that Process2's Interrupt Stack is empty and pointer to this stack is stored in Process2's TSS. Number of the TSS selector, pointing to Process2's TSS, is in the TR registers. Therefore Process2 is prepared to receive interrupts (that will execute on its Interrupt Stack).

Note that Process1's Interrupt Stack contains the current address of Process1's Main Stack, Process1's return address, as well as values of General Purpose registers. A pointer to this stack is stored in the General Process table. Therefore Process1 is fully prepared for a context switch, that will give it control.

Process Creation

To start a new process, the scheduler:

1. Calls the ELF file loader/parser, which allocates the Memory Area of the process and puts code and data (from the ELF file) there.
2. Allocates memory for the TSS and Interrupt Stack.
3. Allocates a File Descriptor Table.
4. Creates GDT entries (CS, DS, TSS selector)
5. Puts on the InterruptStack entry point, code segment, data segment, stack pointer (pointing to the end of the Memory Area allocated in step 1) and initial values of the general purpose registers. Those values will be popped during a context switch (as described above).
6. Creates a new entry in the General Process Table, putting process data (including pointer to the Interrupt Stack, TSS selector number and pointer to the File Descriptor Table) into the Process Descriptor in that table.